DSCI 551 – Spring 2026
Homework 5: Spark DataFrame and RDD

(100 points)

Due: 11:59pm, April 17, 2026, Friday

NO Late Submissions will be accepted!

A. Understanding Spark Join Algorithms via DataFrame Hints [30 points]

In this task, you will explore how Apache Spark chooses different **join algorithms** when working with DataFrames. You will use **join hints** and analyze the **physical execution plan** to understand how Spark executes joins under different conditions.

By completing this task, you will be able to:

- Identify different Spark join algorithms:
  - Broadcast Hash Join
  - Shuffle Hash Join
  - Sort Merge Join
- Interpret Spark's `explain()` output (physical plan)
- Understand how join hints influence execution strategies
- Compare performance trade-offs between join strategies

You will create the following DataFrames using the csv files posted for the Spark DataFrame tutorial.

- `country`
- `city`
- cl (for countrylanguage)

Each DataFrame contains a column that can be used for joins:

- `country.Code`
- `city.CountryCode`
- `cl.CountryCode`

You will run and analyze the following three Spark queries:

```
country.hint('broadcast').join(city, country.Code ==
city.CountryCode).explain()
```

```
country.hint('shuffle_hash').join(city, country.Code ==
city.CountryCode).explain()

cl.hint('merge').join(country, country.Code ==
cl.CountryCode).explain()
```

Answer the following questions:

1. The "Traffic Cop" Question (Broadcast):
   In the first plan, you see a BroadcastExchange. This means Spark is sending a full copy of the country table to every single worker. If the country table was as big as a movie file (several GBs), why would this make the computer slow or cause it to crash?

2. The "Matching Keys" Question (Shuffle):
   In the Shuffle plans, you see hashpartitioning(..., 200). Why does Spark move data around so that rows with the same CountryCode end up in the same bucket? What would happen if 'USA' data was on Worker A and its matching city was on Worker B?".

3. The "Library Book" Question (Sort-Merge):
   In the SortMergeJoin plan, Spark uses a Sort step before joining. Think of a library: is it faster to find a book if the shelves are messy or if they are alphabetized? Why does Spark spend time sorting the data before it starts the join?

4. The "Spot the Difference" Question (Operators):
   Look at the Shuffle Hash Join plan and the Sort-Merge Join plan side-by-side. One of these plans skips a very heavy step, making it faster but more likely to "run out of breath" (RAM). Which specific operator (node) is **missing** from the Shuffle Hash Join plan? Why does skipping this step make the join faster, but also riskier for the computer's memory?

5. The "Safety First" Question (Memory):
   Imagine you have a very small amount of RAM (memory). Which join is 'safer' because it can write temporary data to the hard drive instead of keeping everything in memory: the Hash Join or the Sort-Merge Join?

6. The "Filter" Question (Efficiency):
   In all plans, the Filter (like isnotnull) happens right at the beginning (FileScan). Why is it a good idea to throw away the rows we don't need before we start moving data across the network?

What to submit

- Submit a single document (PDF or DOCX) containing:
  - The physical plans from `.explain()` for all queries
  - Answers to all questions
- Name your file as SparkDF_john_smith.pdf/docx.

B. Mini-RDD: Transformations and Joins [70 points]

Overview

In this task, you will complete several missing pieces of a small Python RDD library (MiniRDD.ipynb). The goal is to practice distributed-data style thinking by implementing transformations and key-based operations using only standard Python data structures and iterators.

You will implement parts of:

- `union`
- `flatMapValues`
- `reduceByKey`
- join-related logic for `join` and outer joins

The implementation should work with the provided `RDD` and `PipelinedRDD` classes.

Learning goals

By completing this task, you should be able to:

- distinguish narrow and wide transformations,
- reason about grouped key/value data,
- implement iterator-based transformations,
- understand how joins can be built from simpler primitives,
- preserve lazy evaluation where appropriate.

Starter code

You are given an `RDD` class with methods such as `map`, `filter`, `groupByKey`, and `collect`. Some methods are complete, while others are partially implemented or left for you to finish.

Your task is to complete the TODO sections in the starter file.

Tasks

1. Implement `union`

Write code so that `rdd1.union(rdd2)` returns all elements from the left RDD followed by all elements from the right RDD.

Expected behavior:

```
RDD.parallelize([1, 2]).union(RDD.parallelize([3, 4])).collect()
# [1, 2, 3, 4]
```

2. Implement `flatMapValues`

For each key-value pair `(k, v)`, apply a function to `v` that returns an iterable, then emit one output pair for each produced value.

Example:

```
RDD.parallelize([("a", [1, 2]), ("b", [3])]).flatMapValues(lambda
xs: xs).collect()
# [("a", 1), ("a", 2), ("b", 3)]
```

3. Implement `reduceByKey`

Given `(key, value)` pairs, combine all values with the same key using the provided binary function.

Example:

```
RDD.parallelize([("a", 1), ("a", 2), ("b",
5)]).reduceByKey(lambda x, y: x + y).collect()
# [("a", 3), ("b", 5)]
```

4. Implement join logic

Complete the join support so that the following operations work correctly:

- `join`
- `leftOuterJoin`
- `rightOuterJoin`
- `fullOuterJoin`

Use the provided helper structure, and build the functions using `groupByKey`, `flatMapValues`, and iterator logic.

Join behavior:

```
RDD.parallelize([(1, "a")]).join(RDD.parallelize([(1,
"x")])).collect()
# [(1, ("a", "x"))]
```

Outer join behavior should use `None` when one side is missing.

Requirements

Your implementation must satisfy the following:

- Preserve the order of `union` as left data followed by right data.
- `flatMapValues` must emit zero or more results per input pair.
- `reduceByKey` must aggregate values by key using the provided function.
- Join operations must handle duplicate keys correctly by producing the Cartesian product of matches.
- Empty inputs must be handled gracefully.
- Your code should remain compatible with `collect()` and the existing `RDD` API.

Constraints

- Do not use Spark.
- Do not use pandas, NumPy, or external distributed libraries.
- Use only Python standard library tools.
- Keep your solution compatible with lazy evaluation in the existing design.
- Do not break existing methods such as `map`, `filter`, `groupByKey`, or `cache`.

Suggested approach

A good way to think about this assignment is:

- `union` is a simple concatenation of two data streams.
- `flatMapValues` applies a function to each value and emits one row per output value.
- `reduceByKey` can be built by scanning grouped key/value pairs and combining values incrementally.
- joins can be implemented by tagging left and right records, grouping by key, and then constructing output pairs from the grouped values.

What to submit

Submit your completed ipynb file, and append your name, e.g., MiniRDD_john_smith.ipynb.

Evaluation

Your work will be graded on:

- correctness,
- handling of edge cases,
- code clarity,
- compatibility with the existing API,
- whether the implementation follows the intended lazy/pipelined design.

Example test cases

Your code should pass all the test cases, attached in the end of the provided ipynb file.